In [5]:
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras import models, layers
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

print("TensorFlow version:", tf.__version__)
```

```
TensorFlow version: 2.22.0-rc0
```

In [6]:
```
from tensorflow.keras.datasets import mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()

print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)

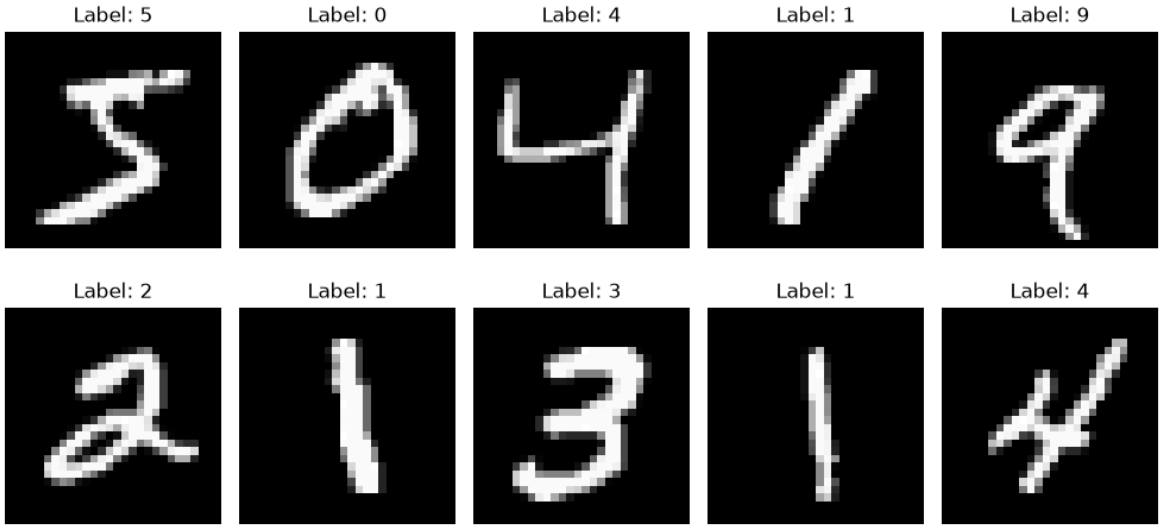
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-dataset
s/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 6s 1us/step
Training images: (60000, 28, 28)
Training labels: (60000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)
```

In [7]:
```
plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```



In [8]:
```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

In [9]:
```
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

print("Training shape:", x_train.shape)
print("Testing shape:", x_test.shape)
```

```
Training shape: (60000, 28, 28, 1)
Testing shape: (10000, 28, 28, 1)
```

In [10]:
```
x_val = x_train[-5000:]
y_val = y_train[-5000:]

x_train = x_train[:-5000]
y_train = y_train[:-5000]

print("Training:", x_train.shape)
print("Validation:", x_val.shape)
print("Testing:", x_test.shape)
```

```
Training: (55000, 28, 28, 1)
Validation: (5000, 28, 28, 1)
Testing: (10000, 28, 28, 1)
```

In [11]:
```
model = models.Sequential([

    layers.Conv2D(
        32,
        (3, 3),
        activation="relu",
        input_shape=(28, 28, 1)
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),
```

```
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(
        128,
        activation="relu"
    ),

    layers.Dense(
        10,
        activation="softmax"
    )
])

model.summary()
```

```
c:\Users\DIKSHA SAINI\OneDrive\Attachments\Desktop\Python\.venv\Lib\site-packages
\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `in
put_shape`/`input_dim` argument to a layer. When using Sequential models, prefer
using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

In [12]:
```
layers.Conv2D(32, (3,3), activation="relu")
```

Out[12]:
```
<Conv2D name=conv2d_2, built=False>
```

In [13]:
```
activation="relu"
```

In [14]:
```
layers.MaxPooling2D((2,2))
```

Out[14]:
```
<MaxPooling2D name=max_pooling2d_2, built=True>
```

In [15]:
```
layers.Conv2D(64, (3,3), activation="relu")
```

Out[15]: <Conv2D name=conv2d_3, built=False>

In [16]:
```
layers.Flatten()
```

Out[16]: <Flatten name=flatten_1, built=False>

In [17]:
```
layers.Dense(128, activation="relu")
```

Out[17]: <Dense name=dense_2, built=False>

In [18]:
```
layers.Dense(10, activation="softmax")
```

Out[18]: <Dense name=dense_3, built=False>

In [19]:
```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

In [20]:
```
history = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=128,
    validation_data=(x_val, y_val)
)
```

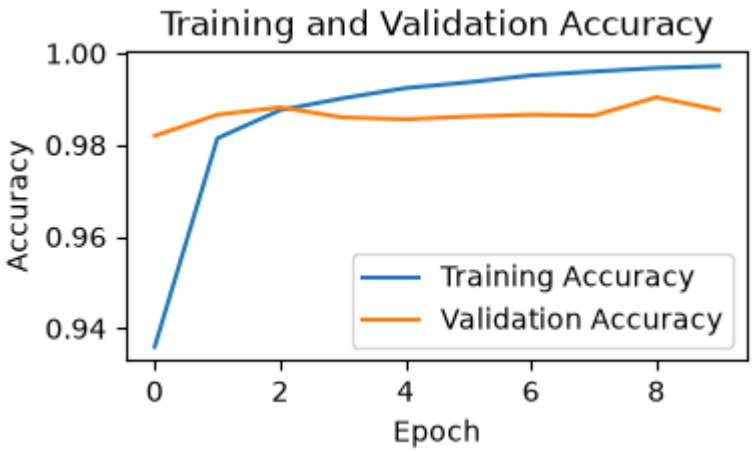
```
Epoch 1/10
430/430 ━━━━━━━━━━━━━━━━━━━━ 14s 32ms/step - accuracy: 0.9360 - loss: 0.2172 - va
l_accuracy: 0.9820 - val_loss: 0.0626
Epoch 2/10
430/430 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9815 - loss: 0.0596 - val
_accuracy: 0.9866 - val_loss: 0.0473
Epoch 3/10
430/430 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9876 - loss: 0.0414 - val
_accuracy: 0.9882 - val_loss: 0.0419
Epoch 4/10
430/430 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9902 - loss: 0.0314 - val
_accuracy: 0.9860 - val_loss: 0.0438
Epoch 5/10
430/430 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9924 - loss: 0.0246 - val
_accuracy: 0.9856 - val_loss: 0.0495
Epoch 6/10
430/430 ━━━━━━━━━━━━━━━━━━━━ 6s 15ms/step - accuracy: 0.9937 - loss: 0.0196 - val
_accuracy: 0.9862 - val_loss: 0.0464
Epoch 7/10
430/430 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9952 - loss: 0.0154 - val
_accuracy: 0.9866 - val_loss: 0.0455
Epoch 8/10
430/430 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9960 - loss: 0.0122 - val
_accuracy: 0.9864 - val_loss: 0.0551
Epoch 9/10
430/430 ━━━━━━━━━━━━━━━━━━━━ 7s 17ms/step - accuracy: 0.9968 - loss: 0.0100 - val
_accuracy: 0.9904 - val_loss: 0.0401
Epoch 10/10
430/430 ━━━━━━━━━━━━━━━━━━━━ 16s 37ms/step - accuracy: 0.9972 - loss: 0.0084 - va
l_accuracy: 0.9876 - val_loss: 0.0565
```

In [23]:
```
plt.figure(figsize=(4, 2))

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()

plt.show()
```

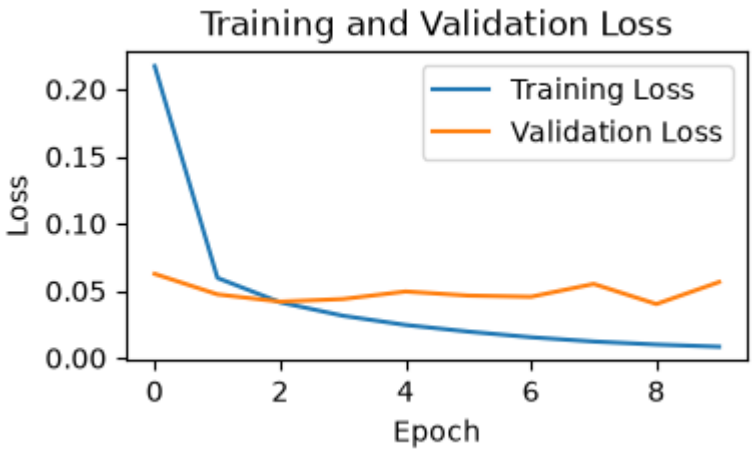


In [25]:
```
plt.figure(figsize=(4, 2))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()

plt.show()
```



In [26]:
```
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=1
)
```

```
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9904 - loss: 0.0349
Test Loss: 0.03488530218601227
Test Accuracy: 0.9904000163078308
```

In [27]:
```
y_probability = model.predict(x_test)

y_pred = np.argmax(y_probability, axis=1)

print("Predicted labels:")
print(y_pred[:20])

print("Actual labels:")
print(y_test[:20])
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
Predicted labels:
[7 2 1 0 4 1 4 9 5 9 0 6 9 0 1 5 9 7 5 4]
Actual labels:
[7 2 1 0 4 1 4 9 5 9 0 6 9 0 1 5 9 7 3 4]
```

In [28]:
```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

```
Accuracy: 0.9904
```

In [29]:
```
precision = precision_score(
    y_test,
    y_pred,
    average="weighted"
)

recall = recall_score(
    y_test,
    y_pred,
    average="weighted"
)

f1 = f1_score(
    y_test,
    y_pred,
    average="weighted"
)

print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)
```

```
Precision: 0.99042977464372
Recall: 0.9904
F1-Score: 0.9903976300376062
```

In [30]:
```
print(
    classification_report(
        y_test,
        y_pred
    )
)
```

	precision	recall	f1-score	support
0	1.00	0.99	0.99	980
1	0.99	1.00	0.99	1135
2	1.00	0.98	0.99	1032
3	0.99	0.99	0.99	1010
4	0.99	0.99	0.99	982
5	0.99	0.99	0.99	892
6	0.98	0.99	0.99	958
7	0.99	0.99	0.99	1028
8	0.99	0.99	0.99	974
9	0.99	0.99	0.99	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

In [31]:
```
cm = confusion_matrix(y_test, y_pred)

print(cm)
```

```
[[ 972    0    0    0    0    0    6    1    1    0]
 [   0 1132    1    1    0    0    1    0    0    0]
 [   0    5 1010    1    1    0    1    7    7    0]
 [   0    0    0 1003    0    4    0    1    2    0]
 [   0    0    0    0  976    0    1    0    0    5]
 [   1    0    0    6    0  881    3    0    0    1]
 [   0    3    0    0    1    2  952    0    0    0]
 [   0    2    1    1    0    0    0 1022    0    2]
 [   2    0    0    0    0    0    5    3  961    3]
 [   1    2    0    3    3    3    0    1    1  995]]
```

In [33]:
```
plt.figure(figsize=(4, 2))

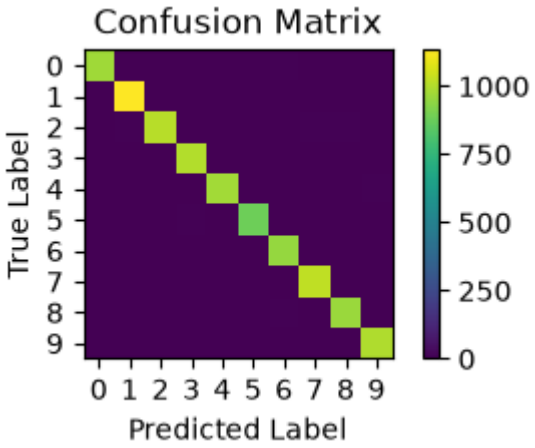
plt.imshow(cm)

plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

plt.colorbar()

plt.xticks(range(10))
plt.yticks(range(10))

plt.show()
```



In [35]:
```
plt.figure(figsize=(6, 4))

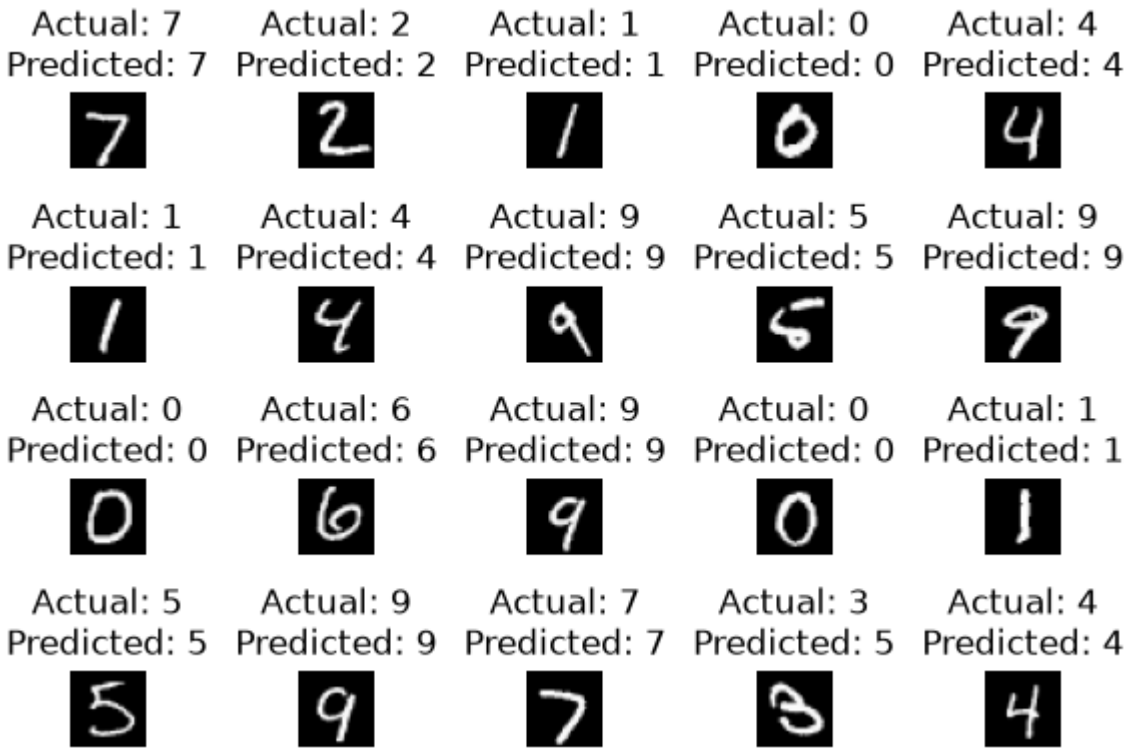
for i in range(20):
    plt.subplot(4, 5, i + 1)

    plt.imshow(
        x_test[i].reshape(28, 28),
        cmap="gray"
    )

    plt.title(
        f"Actual: {y_test[i]}\nPredicted: {y_pred[i]}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```



In [36]:
```
misclassified = np.where(y_pred != y_test)[0]
```

```
print("Number of misclassified images:",
      len(misclassified))
```

```
Number of misclassified images: 96
```

In [38]:
```
plt.figure(figsize=(6, 4))

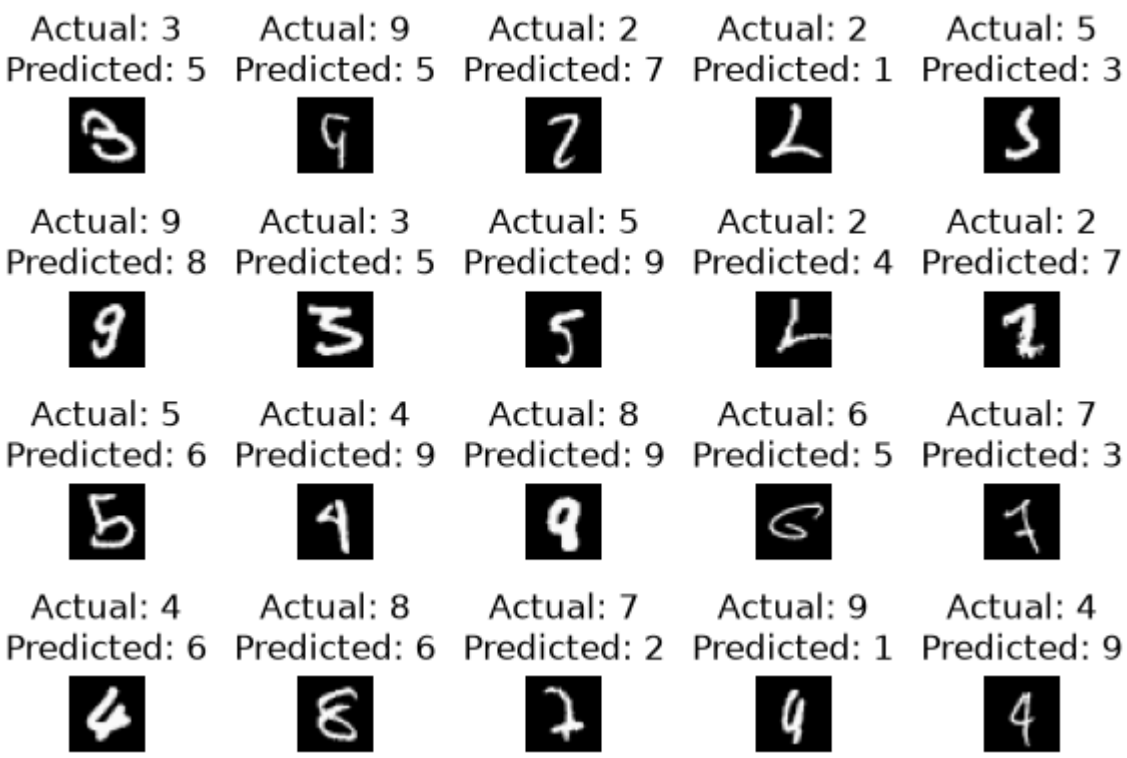
for i, index in enumerate(misclassified[:20]):
    plt.subplot(4, 5, i + 1)

    plt.imshow(
        x_test[index].reshape(28, 28),
        cmap="gray"
    )

    plt.title(
        f"Actual: {y_test[index]}\n"
        f"Predicted: {y_pred[index]}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```



Result: A Convolutional Neural Network was successfully designed and trained for handwritten digit classification using the MNIST dataset. The images were normalized and divided into training, validation, and testing sets. The CNN consisted of convolutional, pooling, flattening, fully connected, and output layers. The model was trained using the Adam optimizer and sparse categorical cross-entropy loss. Its performance was evaluated using accuracy, precision, recall, F1-score, and a confusion matrix. The trained model was also used to predict unseen test images and identify misclassified samples.